

Prática com Docker

Aplicação Web 3-tier Containerizada

Guilherme César Athayde - RA: 748175
UFSCar — DevOps

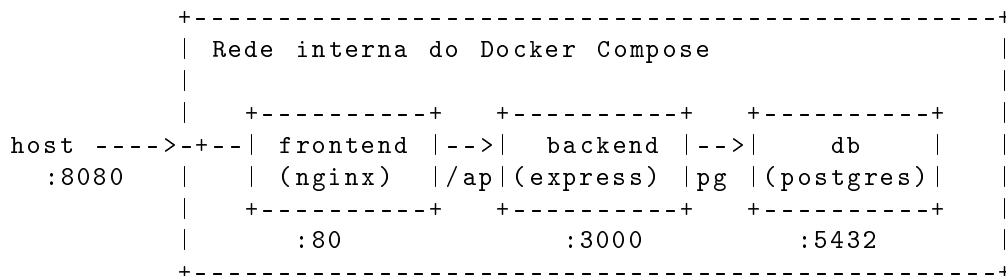
Abril de 2026

1 Visão geral da aplicação

O projeto consiste em uma aplicação web de gerenciamento de tarefas (*to-do list*) implementada sob uma arquitetura clássica de três camadas (*three-tier*), com cada camada executada em um contêiner Docker independente. A aplicação permite ao usuário criar, listar, marcar como concluídas e remover tarefas através de uma interface web, com persistência em banco de dados relacional.

A escolha de uma aplicação 3-tier atende naturalmente ao requisito mínimo de três contêineres da prática, além de demonstrar um caso de uso representativo do mundo real, onde a separação entre apresentação, lógica de negócio e persistência é padrão. A comunicação entre os contêineres ocorre exclusivamente pela rede interna do Docker Compose, utilizando os nomes dos serviços como *hostnames* — nenhum contêiner acessa outro via *localhost* ou porta exportada no *host*.

2 Arquitetura



2.1 Camadas e responsabilidades

frontend Interface de usuário em React (compilada com Vite) servida como conteúdo estático por Nginx. O mesmo Nginx também atua como *reverse-proxy*, encaminhando chamadas */api/** ao backend.

backend API REST em Node.js com Express, que expõe os *endpoints* de CRUD e executa as *queries* no PostgreSQL através da biblioteca *pg*.

db Banco de dados PostgreSQL 16, com armazenamento persistido em volume nomeado do Docker (*db-data*).

2.2 Comunicação entre contêineres

Origem	Destino	Endereço utilizado
frontend (Nginx)	backend	http://backend:3000
backend (Node)	db	db:5432

Apenas o contêiner **frontend** expõe uma porta ao *host* (8080:80). Os contêineres **backend** e **db** não expõem portas externas — permanecem acessíveis somente pela rede interna criada automaticamente pelo Compose, identificados por seus nomes de serviço.

3 Tecnologias utilizadas

Camada	Tecnologia	Papel
Frontend	React 18 + Vite 5	<i>Framework</i> e <i>bundler</i> da SPA
Frontend	Nginx (<i>alpine</i>)	Servidor HTTP + <i>reverse proxy</i>
Backend	Node.js 20 + Express 4	Servidor HTTP e roteamento da API
Backend	pg (node-postgres)	Cliente PostgreSQL
Banco	PostgreSQL 16 (<i>alpine</i>)	Persistência relacional
Orquestr.	Docker + Docker Compose	<i>Build</i> , rede e <i>lifecycle</i>

4 Estrutura do projeto

```
pratica-1/
|-- compose.yml                # Orquestracao dos 3servicos
|-- backend/
|   |-- Dockerfile            # node:20-alpine
|   |-- package.json
|   |-- server.js             # API /api/tasks (CRUD)
|   '-- .dockerignore
'-- frontend/
    |-- Dockerfile            # multi-stage: node (build) -> nginx
    |-- nginx.conf             # SPA + proxy reverso para /api/
    |-- package.json
    |-- vite.config.js
    |-- index.html
    |-- src/
    |   |-- main.jsx
    |   '-- App.jsx
    '-- .dockerignore
```

5 Detalhes dos contêineres

5.1 Contêiner db (PostgreSQL)

Utiliza a imagem oficial `postgres:16-alpine` sem `Dockerfile` próprio. A configuração é feita por variáveis de ambiente no `compose.yml` e a persistência por meio do volume nomeado `db-data`, evitando perda de dados entre reinicializações. Um *healthcheck* com `pg_isready` permite que o backend só seja iniciado após o banco estar pronto.

5.2 Contêiner backend (Node.js + Express)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Listing 1: backend/Dockerfile

O servidor conecta ao PostgreSQL usando o *hostname* `db` (nome do serviço no Compose) através de variáveis de ambiente. Na inicialização, executa um *retry loop* aguardando o banco responder, cria a tabela `tasks` caso não exista, e então passa a escutar na porta 3000.

5.3 Contêiner frontend (React + Nginx)

O Dockerfile do frontend usa **multi-stage build**: a primeira etapa compila a SPA com Node; a segunda copia apenas os artefatos estáticos para uma imagem Nginx enxuta.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 2: frontend/Dockerfile

O arquivo `nginx.conf` configura o servidor para servir a SPA e encaminhar chamadas `/api/` ao backend pelo nome do serviço:

```
location /api/ {
    proxy_pass http://backend:3000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
}
location / {
    try_files $uri $uri/ /index.html;
}
```

Listing 3: frontend/nginx.conf (trecho)

5.4 Arquivo compose.yml

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: tasks
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d tasks"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
```

```

build: ./backend
environment:
  DB_HOST: db
  DB_PORT: 5432
  DB_USER: postgres
  DB_PASSWORD: postgres
  DB_NAME: tasks
  PORT: 3000
depends_on:
  db:
    condition: service_healthy

frontend:
  build: ./frontend
  ports:
    - "8080:80"
  depends_on:
    - backend

volumes:
  db-data:

```

Listing 4: compose.yml

6 Manual de instalação e execução

6.1 Pré-requisitos

- Docker Engine 24+ (ou Docker Desktop) instalado.
- Plugin *Docker Compose* v2 (incluso no Docker Desktop; em Linux pode ser necessário instalar separadamente).
- Portas 8080 do *host* disponível.

6.2 Passo a passo

1. Obtenha o projeto e entre no diretório raiz:

```
cd devops-containerized-3-tier-web
```

2. Construa as imagens e inicie os contêineres:

```
docker compose up --build
```

Na primeira execução, este comando compila a imagem do backend e a imagem multi-stage do frontend (*build* Vite seguido da cópia para a imagem Nginx).

3. Acesse a aplicação no navegador:

```
http://localhost:8080
```

4. Para parar os contêineres, em outro terminal (ou após interromper com Ctrl+C):

```
docker compose down
```

5. Para parar e **apagar** os dados do banco (remove o volume):

```
docker compose down -v
```

6.3 Comandos úteis

Comando	Descrição
<code>docker compose ps</code>	Lista os contêineres em execução.
<code>docker compose logs -f</code>	Acompanha os <i>logs</i> de todos os serviços.
<code>docker compose logs backend</code>	<i>Logs</i> apenas do backend.
<code>docker compose exec db psql -U postgres tasks</code>	Abre o <i>psql</i> dentro do contêiner do banco.
<code>docker compose build --no-cache</code>	Reconstrói imagens do zero.

7 Endpoints da API

Todos os *endpoints* são acessados pelo usuário via proxy reverso em <http://localhost:8080/api/...>

Método	Rota	Descrição
GET	<code>/api/health</code>	<i>Health check</i> da API.
GET	<code>/api/tasks</code>	Lista todas as tarefas.
POST	<code>/api/tasks</code>	Cria uma nova tarefa (corpo: <code>{"title": "..."}).</code>
PATCH	<code>/api/tasks/:id</code>	Altera o estado <code>done</code> da tarefa.
DELETE	<code>/api/tasks/:id</code>	Remove a tarefa.

8 Atendimento ao requisito do enunciado

- **Mínimo de 3 contêineres:** atendido com `frontend`, `backend` e `db`.
- **Dockerfiles individuais:** `backend/Dockerfile` e `frontend/Dockerfile` (o `db` usa a imagem oficial `postgres:16-alpine` diretamente).
- **Arquivo `compose.yml`:** orquestra os três serviços, rede, volume e *healthcheck*.
- **Comunicação por nome de serviço:** o Nginx acessa o backend via `http://backend:3000`, e o backend acessa o banco via `host: db` — nenhum dos dois utiliza `localhost` ou porta exportada no *host*.